SBA301 – SLOT 30

CHAPTER 18 – HIBERNATE APPLICATION DEVELOPMENT & CRUD OPERATIONS

PART A: HIBERNATE CRUD APPLICATION • LAYERS • CONFIGURATION

CORE: Tư duy trung tâm của Slot 30: Một CRUD application tốt không phải là 4 nút Create/Read/Update/Delete, mà là một chuỗi trách nhiệm rõ ràng từ cấu hình Hibernate → entity mapping → DAO → Repository → Service → consumer. Mỗi CRUD operation phải có transaction boundary, error handling, validation, verification và evidence ở database.

Thông tin	Giá trị
Môn học	SBA301 – Integrate Single Page Application with Spring Boot
Slot	30
Chủ đề	Chapter 18 Part A – Hibernate Application Development & CRUD Operations: CRUD application/layers/config
CLO chính	CLO5 – Develop layered Hibernate applications and implement reliable CRUD operations
Ngữ liệu xuyên suốt	Library Management – Student/Book – Chapter 18 + Lab 07 baseline
Course baseline	JDK 21 • Maven • Hibernate course version • SQL Server • IntelliJ
Thời lượng gợi ý	135 phút + implementation/self-study
Learning flow	Prepare → Architect → Configure → Implement CRUD → Layer → Break → Diagnose → Verify → Transfer

NOTE: Scope lock: Slot 30 Part A tập trung application architecture, configuration và CRUD persistence. JavaFX/FXML/StudentController thuộc phần tiếp theo của Chapter 18; chỉ xuất hiện như downstream consumer để sinh viên hiểu dependency direction.

CÁCH SỬ DỤNG GIÁO TRÌNH

Phần IV là knowledge base lý thuyết; Phần VI là workbook triển khai. Mỗi CRUD operation cần được kiểm bằng ít nhất ba góc nhìn: Java state, Hibernate SQL/transaction, và database state.

Pha	Hoạt động	Evidence
Prepare	Chạy baseline Hibernate + LibraryDB.	Build/DB evidence
Architect	Vẽ layer + dependency direction.	Architecture map
Configure	Hibernate config, mappings, SessionFactory.	Bootstrap evidence
Implement	Create/Read/Update/Delete.	DAO/Repository/Service source
Break	Invalid input, missing ID, duplicate email, transaction failure.	Negative evidence
Diagnose	SQL log, exception, rollback, entity state.	Debug note
Verify	DB rows + automated/manual tests.	CRUD matrix
Transfer	Bridge sang JavaFX Part B / framework-managed persistence.	Design note

0. SINH VIÊN CẦN CHUẨN BỊ GÌ TRƯỚC KHI HỌC SLOT

0.1. Kiến thức prerequisite

- Slot 28: SessionFactory, Session, transaction, entity lifecycle, HQL.
- Slot 29: DAO → Repository → Service, many-to-many mapping, transaction/session ownership.
- SQL Server: PK/IDENTITY, UNIQUE, FK, SELECT/INSERT/UPDATE/DELETE.
- Java: exception handling, Optional, collections, interfaces, dependency composition.
- Maven: dependencies, compile/test lifecycle.

0.2. Pre-flight checklist

Check	PASS condition
JDK/Maven	`java -version`, `mvn -v` đúng baseline
LibraryDB	kết nối được và có quyền DDL/DML phù hợp lab
Hibernate project	SessionFactory bootstrap thành công
Entities	Student/Book compile
Git	working tree sạch hoặc checkpoint rõ

Baseline commands

```
java -version
mvn -v
mvn -q -DskipTests compile
mvn test
git status
```

0.3. Pre-class 3–2–1

3 điều đã biết	2 điều còn mơ hồ	1 câu hỏi cần chứng minh
DAO dùng Hibernate; Repository che persistence; Service chứa use-case.	CRUD update nên merge hay dirty checking? Transaction nên nằm DAO hay Service?	Nếu UPDATE fail giữa transaction thì DB có bị partial change không?

I. TÓM TẮT CÁC CHỦ ĐỀ & MỤC TIÊU TRONG BÀI HỌC

1.1. Chapter 18 Part A progression

Nguồn Chapter 18	Slot 30 interpretation
Create project/structure	Composition root + packages/layers
Create Student/Book	Entity + mapping contract
StudentDAO save/get/delete/find/update	CRUD persistence core
IStudentRepository/StudentRepository	Persistence abstraction
IStudentService/StudentService	Use-case boundary
module-info.java	Java module/runtime dependency awareness
hibernate.cfg.xml	Database/runtime configuration

1.2. Learning objectives

1. Thiết kế package/layer structure cho Hibernate CRUD application.
2. Phân biệt configuration/bootstrap, persistence, repository và service responsibilities.
3. Cài đặt Create/Read/Update/Delete có transaction/rollback đúng.
4. Giải thích khi nào entity managed được dirty-check và khi nào cần merge.
5. Thiết kế `findById`, `findAll`, `findByEmail` với typed HQL/Optional phù hợp.
6. Xử lý not-found, duplicate/constraint và invalid input mà không biến mọi lỗi thành RuntimeException mơ hồ.
7. Trace một CRUD request từ consumer xuống DB và ngược lại.
8. Viết CRUD verification matrix dựa trên SQL và DB state.
9. Nhận biết anti-pattern: UI gọi DAO, DAO giữ Session singleton, service chứa HQL, transaction phân mảnh.
10. Chuẩn bị architecture để JavaFX Part B chỉ đóng vai trò presentation layer.

1.3. Mental model

```
Application Consumer (Main / JavaFX later)
        |
        v
Service  ---- business validation / use case
        |
        v
Repository ---- persistence contract
        |
        v
DAO ---- Session / Transaction / HQL / CRUD
        |
        v
SessionFactory -> Session -> JDBC -> SQL Server
        ^
        |
hibernate.cfg.xml + entity mappings
```

1.4. CRUD semantic contract

Operation	Input	Success	Failure examples

Operation	Input	Success	Failure examples
CREATE	new Student	generated ID + row	duplicate email / invalid fields
READ ONE	id	Student	not found
READ LIST	filters/paging	List<Student>	query/config failure
UPDATE	existing identity + changes	updated row	not found / stale/concurrency
DELETE	id	row removed	not found / FK conflict

II. TRỌNG TÂM CỐT LÕI & KỸ NĂNG SINH VIÊN CẦN NẮM VỮNG

2.1. Configuration is runtime infrastructure

`hibernate.cfg.xml` hoặc programmatic config xác định DB connectivity, dialect, mappings và schema behavior.

2.2. SessionFactory lifecycle

Build một lần cho application/configuration; không tạo lại mỗi CRUD.

2.3. Session lifecycle

Short-lived unit-of-work; không share xuyên thread/UI lifetime.

2.4. Transaction boundary

Write CRUD phải atomic; rollback khi exception.

2.5. CREATE

`persist()` chuyển transient entity thành managed; generated ID semantics phụ thuộc strategy.

2.6. READ

`get()` phù hợp PK; HQL phù hợp semantic queries; typed queries giảm cast risk.

2.7. UPDATE – managed path

Load entity trong transaction, sửa field, commit; dirty checking tạo UPDATE.

2.8. UPDATE – detached path

`merge()` copy state vào managed instance và trả managed reference.

2.9. DELETE

Load managed entity rồi `remove()`; FK/association constraints phải được xem xét.

2.10. Not-found semantics

DAO có thể trả Optional; Service chuyển thành application/domain error phù hợp.

2.11. Validation boundary

Entity annotations/schema constraints không thay thế use-case validation.

2.12. DAO

Chứa Hibernate-specific persistence operations.

2.13. Repository

Định nghĩa contract cho persistence theo domain/application.

2.14. Service

Tổ chức business validation/orchestration và chuẩn bị transaction ownership ở cấp use-case.

2.15. Configuration separation

Không hard-code DB password trong Java source/Git.

2.16. Exception strategy

Rollback technical transaction; upper layer nhận error có nghĩa.

2.17. Logging

SQL log hữu ích để học; không log credential/sensitive payload.

2.18. CRUD test discipline

Mỗi happy path phải có negative/not-found/constraint counterpart.

2.19. Idempotency awareness

READ idempotent; DELETE thường mong muốn idempotent semantics ở API layer nhưng DAO boolean/not-found contract cần rõ.

2.20. Layer dependency

Upper layer depends on abstractions; persistence layer không phụ thuộc UI.

2.21. Responsibility matrix

Concern	DAO	Repository	Service	UI/Main
Hibernate Session/HQL	Yes	No/hidden	No	No
CRUD persistence	Yes	Expose contract	Call	No direct
Business validation	No	No	Yes	Input pre-check only
Transaction orchestration	Low-level/course	Can participate	Preferred use-case boundary	No
Presentation/input	No	No	No	Yes

III. ỨNG DỤNG THỰC TIỄN KIẾN THỨC CỦA BÀI HỌC TRONG SẢN PHẨM THỰC TẾ

3.1. Library Management CRUD

Use case	Service rule	Persistence work
Create Student	email required/unique, marks range	INSERT STUDENTS
List Students	sort/filter/page	SELECT/HQL
Get Student	id exists	SELECT by PK
Update Student	exists + fields valid	dirty checking/merge UPDATE
Delete Student	exists + association rule	DELETE + FK handling

3.2. Why CRUD layers matter

Khi chuyển sang JavaFX ở phần sau, Controller chỉ nên gọi `StudentService`; nếu Controller biết `Session`, `Transaction` hoặc HQL thì presentation và persistence đã bị coupling.

3.3. Course vs production

Course baseline	Production direction
hibernate.cfg.xml	externalized config / framework-managed datasource
manual Session/Transaction	managed transaction boundary
hbm2ddl update trong lab	Flyway/Liquibase + validate
manual Repository	Spring Data/Jakarta Data/Hibernate Data where appropriate
console SQL inspection	structured observability + DB slow-query monitoring

IV. NỘI DUNG CHI TIẾT – KNOWLEDGE BASE HIBERNATE CRUD APPLICATION

READING GUIDE: Phần IV giải thích WHY/WHAT. Không dùng như tutorial từng bước.

4.1. CRUD application architecture

- CRUD là use-case flow qua nhiều layer, không phải collection các method độc lập.
- Layering giúp tách persistence mechanics khỏi business/presentation.

4.2. Project/package structure

- entity/model, dao, repository, service, util/config, app/controller là các concern khác nhau.
- Package structure nên phản ánh dependency boundaries.

4.3. Hibernate bootstrap
- Configuration đọc properties/mappings và build SessionFactory.
- Bootstrap failure thường là config/driver/mapping problem, không phải CRUD problem.

4.4. Database configuration
- URL, driver, username/password, dialect, schema-generation options.
- Credential phải externalize ở production.

4.5. Mapping contract
- Entity mapping là contract giữa Java model và schema.
- Nullability/unique/length cần đồng bộ với DB migration/schema.

4.6. SessionFactory as application infrastructure
- Expensive/long-lived/thread-safe.
- One-per-configuration/database là common architecture.

4.7. Session as unit of work
- Short-lived, not thread-safe.
- First-level cache và entity tracking nằm trong Session.

4.8. Transaction semantics
- Commit atomic changes; rollback on failure.
- Flush có thể xảy ra trước commit.

4.9. Create lifecycle
- new Student = transient.
- persist → managed; SQL INSERT có thể phát ở persist/flush/commit tùy ID strategy.

4.10. Read by primary key
- `session.get()` trả entity/null trong native API style.
- PK lookup được first-level cache hỗ trợ.

4.11. Read list with HQL
- Typed HQL `from Student s order by s.id`.
- Production list cần paging/filtering khi data tăng.

4.12. Update via dirty checking
- Load managed entity, mutate, commit.
- Đây là model tự nhiên nhất trong one-unit-of-work.

4.13. Update via merge
- Detached input được merge vào managed copy.
- Phải sử dụng return value nếu tiếp tục thao tác trong context.

4.14. Delete semantics
- remove managed entity.
- Association/FK constraints vẫn quyết định DB success.

4.15. Optional and not-found
- Optional biểu diễn absence rõ hơn null ở repository/service contract.
- Không lạm dụng Optional trong entity fields.

4.16. Uniqueness and duplicate create

- Application pre-check có thể cải thiện UX nhưng DB UNIQUE constraint mới chống race condition cuối cùng.
- Constraint violation phải map thành error có nghĩa.

4.17. Field validation

- Required/format/range ở Service/application boundary.
- DB constraint là safety net, không phải UX validation.

4.18. DAO transaction pattern

- open Session → begin tx → operation → commit; catch → rollback; finally/try-resource → close.
- Read-only simple reads vẫn cần session lifecycle rõ.

4.19. Repository abstraction

- Repository interface tách Service khỏi DAO/Hibernate.
- Method naming nên có semantic intent thay vì expose HQL.

4.20. Service orchestration

- Service chứa validation/not-found/use-case.
- Multi-step operations ideally share one transaction boundary.

4.21. Composition root

- Main/bootstrap tạo SessionFactory, DAO, Repository, Service.
- Tránh new DAO rải rác trong controller.

4.22. Module system awareness

- Chapter source có module-info.java vì JavaFX/module setup.
- Part A chỉ cần hiểu dependencies/exports/opens nếu dùng JPMS.

4.23. Exception categories

- ConfigurationException/bootstrap; PersistenceException/Hibernate exception; ConstraintViolation/SQL errors; domain/application errors.
- Chẩn đoán theo layer để tránh catch Exception chung chung.

4.24. Rollback correctness

- Sau transaction exception, rollback rồi không tiếp tục reuse state tùy failure.
- Không swallow exception sau rollback.

4.25. Resource management

- Session phải close mọi path.
- SessionFactory close khi application shutdown.

4.26. HQL parameters

- Bind parameters thay concatenate.
- Query input validation và authorization vẫn cần riêng.

4.27. Pagination

- setFirstResult/setMaxResults + deterministic order.
- Không load toàn bộ table vào JavaFX TableView khi data lớn.

4.28. Search

- Repository method như findByEmail/searchByName giấu query mechanics.
- Search semantics/case-sensitivity phụ thuộc DB/collation/query.

4.29. Update partial vs full

- Entity merge từ form đầy đủ có risk overwrite stale fields.
- Prefer load-and-apply allowed changes trong transaction khi business rules quan trọng.

4.30. Concurrency awareness

- Read-modify-write có lost-update risk.
- @Version optimistic locking là production extension.

4.31. Association-aware CRUD

- Delete Student có books cần relation handling.
- CRUD đơn entity không thể bỏ qua FK/join table.

4.32. Schema generation

- create/update tiện lab.
- Migration tool phù hợp production governance.

4.33. Logging generated SQL

- show_sql/format_sql hỗ trợ học.
- Bind value logging có thể lộ sensitive data; production policy cần cẩn trọng.

4.34. Testing DAO

- Integration test cần DB state thật hoặc compatible test environment.
- Mock Session không chứng minh mapping/SQL đúng.

4.35. Testing Service

- Mock Repository để test validation/orchestration.
- Integration test để chứng minh transaction/persistence.

4.36. Error feedback

- Service trả/throw meaningful error; presentation layer biến thành message.
- Không hiện stack trace raw cho end user.

4.37. CRUD anti-patterns

- DAO singleton Session, UI gọi Session, service chứa HQL, catch-all returning null, commit mỗi line, hard-coded password, `hbm2ddl=update` production.

4.38. Bridge to JavaFX Part B

- StudentController gọi StudentService.
- Form validation + TableView refresh là presentation concerns.

4.39. Current Hibernate direction

- Hibernate ORM 7.4 là latest stable vào 2026-10-12; 8.0 vẫn development.
- Course giữ version baseline hiện tại.

4.40. Compatibility awareness

- Hibernate 7.4 hỗ trợ Java 17/21/25/26 và Jakarta Persistence 3.2 theo official release page.
- Version upgrade là migration project, không phải copy dependency number.

V. KINH NGHIỆM THỰC TIỄN, TIPS

Lỗi	Triệu chứng	Nguyên nhân	Cách xử lý
Bootstrap fail	App không start	URL/driver/mapping	đọc root cause
CREATE duplicate	constraint error	email unique	validate + DB constraint

Lỗi	Triệu chứng	Nguyên nhân	Cách xử lý
UPDATE no effect	DB không đổi	detached/no tx	managed/merge + commit
DELETE fails	FK conflict	association tồn tại	business/delete policy
Session leak	pool/resource issue	không close	try-with-resources
Partial data	một bước commit, bước sau fail	transaction split	move boundary up
UI freeze later	CRUD DB on UI thread	blocking I/O	Part B async/threading awareness
Service returns null	NPE downstream	not-found contract mơ hồ	Optional/domain exception

5.1. Debug decision tree

```
CRUD failure
|
+-- Application doesn't start?
|   +-- hibernate.cfg.xml / driver / credentials / mapping
|
+-- CREATE fails?
|   +-- validation?
|   +-- unique/FK constraint?
|   +-- tx rollback?
|
+-- UPDATE not persisted?
|   +-- managed or detached?
|   +-- merge return used?
|   +-- transaction committed?
|
+-- DELETE fails?
|   +-- entity exists?
|   +-- association/FK?
|
+-- Data partially changed?
    +-- transaction split across DAO calls?
```

- Luôn kiểm DB state sau CRUD, không chỉ message console.
- Không tạo SessionFactory trong DAO constructor nếu mỗi DAO làm một instance riêng.
- Create/update/delete phải có rollback path.
- Service validation không thay DB constraint; DB constraint không thay Service validation.
- Read list phải chuẩn bị paging từ sớm.
- Không trả `null` cho mọi error; phân biệt not-found và technical failure.

VI. HƯỚNG DẪN STEP BY STEP THỰC BÀI TẬP CHỦ ĐỀ VÀ BÀI TẬP TỔNG HỢP HỆ THỐNG

WORKBOOK RULE: Mỗi bước: Mục tiêu → thao tác → code/config → command → verification → PASS/FAIL.

6.1. Step 1 – Tạo/chuẩn hóa project structure

```
src/main/java/fu/edu/sba301/ch18/
├─ entity/
│  ├─ Student.java
│  └─ Book.java
├─ dao/
│  └─ StudentDAO.java
├─ repository/
│  ├─ IStudentRepository.java
│  └─ StudentRepository.java
├─ service/
│  ├─ IStudentService.java
│  └─ StudentService.java
├─ util/
│  └─ HibernateUtil.java
└─ app/
```

```
   └─ Main.java

src/main/resources/
└─ hibernate.cfg.xml
```

PASS: persistence code không nằm trong `Main`/future JavaFX controller.

6.2. Step 2 – Hibernate configuration

hibernate.cfg.xml

```
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.driver_class">
      com.microsoft.sqlserver.jdbc.SQLServerDriver
    </property>
    <property name="hibernate.connection.url">
      jdbc:sqlserver://localhost:1433;
      databaseName=LibraryDB;
      encrypt=true;
      trustServerCertificate=true
    </property>
    <property name="hibernate.connection.username">${DB_USER}</property>
    <property name="hibernate.connection.password">${DB_PASSWORD}</property>
    <property name="hibernate.dialect">
      org.hibernate.dialect.SQLServerDialect
    </property>
    <property name="hibernate.show_sql">true</property>
    <property name="hibernate.format_sql">true</property>
    <property name="hibernate.hbm2ddl.auto">update</property>

    <mapping class="fu.edu.sba301.ch18.entity.Student"/>
    <mapping class="fu.edu.sba301.ch18.entity.Book"/>
  </session-factory>
</hibernate-configuration>
```

SECURITY: `${...}` placeholder cần cơ chế resolve phù hợp project; không commit credential thật.

6.3. Step 3 – SessionFactory utility

```
public final class HibernateUtil {
    private static final SessionFactory SESSION_FACTORY =
        new Configuration()
            .configure("hibernate.cfg.xml")
            .buildSessionFactory();

    public static SessionFactory getSessionFactory() {
        return SESSION_FACTORY;
    }

    public static void shutdown() {
        SESSION_FACTORY.close();
    }

    private HibernateUtil() {}
}
```

PASS: factory build một lần.

6.4. Step 4 – Student entity

```
@Entity
@Table(name = "STUDENTS")
public class Student {
    @Id
```

```
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 120)
    private String email;

    @Column(nullable = false, length = 80)
    private String firstName;

    @Column(nullable = false, length = 80)
    private String lastName;

    private double marks;

    public Student() {}
    // constructor/getters/setters
}
```

6.5. Step 5 – CREATE DAO

```
public Student save(Student student) {
    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        try {
            session.persist(student);
            tx.commit();
            return student;
        } catch (RuntimeException ex) {
            tx.rollback();
            throw ex;
        }
    }
}
```

Verification

```
SELECT TOP 20 * FROM dbo.STUDENTS ORDER BY id DESC;
```

PASS: generated ID + row tồn tại.

6.6. Step 6 – READ ALL DAO

```
public List<Student> findAll() {
    try (Session session = sessionFactory.openSession()) {
        return session.createQuery(
            "from Student s order by s.id",
            Student.class
        ).getResultList();
    }
}
```

PASS: typed query + deterministic order.

6.7. Step 7 – READ ONE DAO

```
public Optional<Student> findById(Long id) {
    try (Session session = sessionFactory.openSession()) {
        return Optional.ofNullable(
            session.get(Student.class, id)
        );
    }
}
```

PASS: existing ID → present; missing ID → Optional.empty().

6.8. Step 8 – SEARCH DAO

```
public Optional<Student> findByEmail(String email) {
    try (Session session = sessionFactory.openSession()) {
        return session.createQuery(
            "from Student s where s.email = :email",
            Student.class
        )
        .setParameter("email", email)
        .uniqueResultOptional();
    }
}
```

6.9. Step 9 – UPDATE managed path

```
public Student updateMarks(Long id, double marks) {
    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        try {
            Student s = session.get(Student.class, id);
            if (s == null) {
                tx.rollback();
                throw new IllegalArgumentException("Student not found");
            }
            s.setMarks(marks); // dirty checking
            tx.commit();
            return s;
        } catch (RuntimeException ex) {
            if (tx.isActive()) tx.rollback();
            throw ex;
        }
    }
}
```

6.10. Step 10 – UPDATE detached path

```
public Student update(Student detached) {
    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        try {
            Student managed = session.merge(detached);
            tx.commit();
            return managed;
        } catch (RuntimeException ex) {
            tx.rollback();
            throw ex;
        }
    }
}
```

IMPORTANT: Không bỏ qua return value của `merge()`.

6.11. Step 11 – DELETE DAO

```
public boolean delete(Long id) {
    try (Session session = sessionFactory.openSession()) {
        Transaction tx = session.beginTransaction();
        try {
            Student s = session.get(Student.class, id);
            if (s == null) {
                tx.rollback();
                return false;
            }
```

```
            session.remove(s);
            tx.commit();
            return true;
        } catch (RuntimeException ex) {
            if (tx.isActive()) tx.rollback();
            throw ex;
        }
    }
}
```

6.12. Step 12 – Repository contract

```
public interface IStudentRepository {
    Student save(Student student);
    List<Student> findAll();
    Optional<Student> findById(Long id);
    Optional<Student> findByEmail(String email);
    Student update(Student student);
    boolean delete(Long id);
}
```

6.13. Step 13 – Repository implementation

```
public class StudentRepository implements IStudentRepository {
    private final StudentDAO dao;

    public StudentRepository(StudentDAO dao) {
        this.dao = dao;
    }

    public Student save(Student s) { return dao.save(s); }
    public List<Student> findAll() { return dao.findAll(); }
    public Optional<Student> findById(Long id) { return dao.findById(id); }
    public Optional<Student> findByEmail(String email) { return dao.findByEmail(email); }
    public Student update(Student s) { return dao.update(s); }
    public boolean delete(Long id) { return dao.delete(id); }
}
```

6.14. Step 14 – Service contract

```
public interface IStudentService {
    Student create(Student student);
    List<Student> getAll();
    Student getById(Long id);
    Student update(Student student);
    void delete(Long id);
}
```

6.15. Step 15 – Service validation/business boundary

```
public class StudentService implements IStudentService {
    private final IStudentRepository repository;

    public StudentService(IStudentRepository repository) {
        this.repository = repository;
    }

    public Student create(Student s) {
        validate(s);

        repository.findByEmail(s.getEmail()).ifPresent(existing -> {
            throw new IllegalStateException("Email already exists");
```

```
        });

        return repository.save(s);
    }

    public Student getById(Long id) {
        return repository.findById(id)
            .orElseThrow(() ->
                new IllegalArgumentException("Student not found"));
    }

    // getAll/update/delete ...

    private void validate(Student s) {
        if (s == null) throw new IllegalArgumentException("Student is required");
        if (s.getEmail() == null || s.getEmail().isBlank())
            throw new IllegalArgumentException("Email is required");
        if (s.getMarks() < 0 || s.getMarks() > 10)
            throw new IllegalArgumentException("Marks must be 0..10");
    }
}
```

6.16. Step 16 – Composition root / Main

```
public class Main {
    public static void main(String[] args) {
        SessionFactory sf = HibernateUtil.getSessionFactory();

        StudentDAO dao = new StudentDAO(sf);
        IStudentRepository repo = new StudentRepository(dao);
        IStudentService service = new StudentService(repo);

        Student created = service.create(
            new Student("slot30@fu.edu.vn", "Slot", "Thirty", 8.5)
        );

        System.out.println(service.getById(created.getId()));
        service.getAll().forEach(System.out::println);

        HibernateUtil.shutdown();
    }
}
```

6.17. Step 17 – CREATE negative test: duplicate email

Gọi `service.create()` hai lần với cùng email.

Expected: application validation/pre-check hoặc DB UNIQUE constraint từ chối; transaction rollback; chỉ một row.

```
SELECT email, COUNT(*) AS cnt
FROM STUDENTS
GROUP BY email
HAVING COUNT(*) > 1;
```

PASS: không có duplicate.

6.18. Step 18 – READ negative test: missing ID

Gọi `service.getById(999999L)`.

Expected: meaningful not-found exception/message, không NullPointerException.

6.19. Step 19 – UPDATE negative test

Test marks = -1, 12 và ID không tồn tại. Service phải từ chối invalid data trước persistence khi phù hợp.

6.20. Step 20 – DELETE + FK awareness

Nếu Student có association Books từ Slot 29, test delete Student và quan sát join rows/FK. Xác nhận shared Book không bị xóa ngoài ý muốn.

6.21. Step 21 – Transaction rollback experiment

```
try (Session session = sf.openSession()) {
    Transaction tx = session.beginTransaction();
    try {
        session.persist(new Student("a@fu.edu.vn","A","A",8));
        session.persist(new Student("a@fu.edu.vn","B","B",9)); // duplicate
        tx.commit();
    } catch (RuntimeException ex) {
        tx.rollback();
    }
}
```

Expected: không partial commit trong transaction.

6.22. Step 22 – Paging DAO extension

```
public List<Student> findPage(int offset, int limit) {
    try (Session session = sessionFactory.openSession()) {
        return session.createQuery(
            "from Student s order by s.id",
            Student.class
        )
        .setFirstResult(offset)
        .setMaxResults(limit)
        .getResultList();
    }
}
```

6.23. Step 23 – Break-It: Session singleton

Cố ý giữ một Session field dùng lại cho nhiều operations/threads; quan sát stale context/resource/thread-safety problems. Sau đó refactor về short-lived session.

BREAK-IT: Không merge anti-pattern này.

6.24. Step 24 – Break-It: commit từng DAO step

Mô phỏng Service use-case hai thay đổi nhưng mỗi DAO method tự commit. Làm bước hai fail và quan sát partial state. Đây là evidence cho việc transaction boundary cần nâng lên ở use-case khi nghiệp vụ multi-step.

6.25. Step 25 – CRUD test matrix

ID	Case	Expected
C01	Create valid Student	row + generated id
C02	Create duplicate email	rejected; no duplicate row
R01	Get existing ID	Student
R02	Get missing ID	not-found
R03	List students	ordered list
U01	Update valid marks	row updated
U02	Update invalid marks	rejected
U03	Update missing ID	not-found
D01	Delete existing	row removed
D02	Delete missing	defined not-found/false
T01	2 writes + second fails	rollback atomic
L01	UI/Main only calls Service	layering pass

6.26. Step 26 – Integration Gate G1–G14

Gate	Evidence
G1 Build	compile/tests pass
G2 Config	Hibernate connects LibraryDB
G3 Bootstrap	SessionFactory single lifecycle
G4 Entity	Student mapping correct
G5 Create	INSERT verified

Gate	Evidence
G6 Read	get/list/search verified
G7 Update	dirty checking/merge verified
G8 Delete	DELETE/FK behavior verified
G9 Rollback	no partial transaction
G10 DAO	Hibernate-specific code isolated
G11 Repository	contract hides DAO
G12 Service	validation/use-case boundary
G13 Negative	duplicate/not-found/invalid cases
G14 Explain	managed update vs detached merge

6.27. Bài tập tổng hợp

1. Hoàn thiện cấu trúc project Chapter 18 Part A.
2. Chuẩn hóa `hibernate.cfg.xml` không hard-code credential trong source.
3. Implement StudentDAO 5 CRUD operations + findByEmail.
4. Implement Repository interface/implementation.
5. Implement Service validation + not-found handling.
6. Chạy full CRUD flow trong Main.
7. Thêm paging.
8. Kiểm transaction rollback với intentional failure.
9. Kiểm delete khi có Student–Book association.
10. Tạo CRUD verification report với SQL evidence.
11. Viết architecture note chuẩn bị cho JavaFX Controller ở Part B.

VII. TỪ ĐIỂN THUẬT NGỮ – FOUNDATION GLOSSARY

Thuật ngữ	Định nghĩa	Ví dụ Slot 30
CRUD	Create Read Update Delete	Student operations
Bootstrap	Khởi tạo Hibernate runtime	Configuration→SessionFactory
SessionFactory	Long-lived factory	one application config
Session	Unit of work	DAO operation
Transaction	Atomic DB work	commit/rollback
DAO	Hibernate persistence implementation	StudentDAO
Repository	Persistence contract	IStudentRepository
Service	Use-case/business layer	StudentService
Composition Root	Nơi wire dependencies	Main
Dirty Checking	Auto-detect managed changes	marks update
Merge	Copy detached state to managed	update detached
Optional	Explicit absence	findById
Constraint	DB rule	UNIQUE email
Rollback	Undo transaction	duplicate failure
Typed HQL	Query có result type	Student.class
Named Parameter	Bound HQL value	:email
Pagination	Windowed result	offset/limit
Layer Violation	Dependency sai	Controller→Session
Schema Generation	Hibernate DDL behavior	hbm2ddl
Migration	Controlled schema change	Flyway/Liquibase

VIII. RESEARCH BASE VÀ AI-BASED LEARNING – 4 NHÓM TRẢI NGHIỆM CUỐI BUỔI

8.1. Quy tắc

- AI không được bịa method/class ngoài source project.
- Mọi CRUD/lifecycle claim phải kiểm SQL/DB/test.
- Không gửi credential thật.
- Current-version claim phải có official Hibernate source + access date.
- Phân biệt Course Baseline và Current Production Direction.

8.2. NHÓM I – Workflow / execution rule

PROMPT I.1: Trace CREATE từ Main → Service → Repository → DAO → Session.persist → SQL INSERT → generated id.

PROMPT I.2: Trace READ by id và first-level cache nếu gọi hai lần trong cùng Session.

PROMPT I.3: Trace managed UPDATE không gọi explicit update nhưng phát SQL ở flush/commit.

PROMPT I.4: Trace detached entity → merge và object reference nào managed.

PROMPT I.5: Trace DELETE có Student–Book association và FK/join table.

PROMPT I.6: Trace duplicate email từ Service pre-check tới DB constraint fallback.

PROMPT I.7: Trace transaction rollback khi operation thứ hai fail.

PROMPT I.8: Trace SessionFactory bootstrap từ cfg/mapping tới connection.

PROMPT I.9: Trace findAll paging từ HQL tới SQL Server limit/offset translation.

PROMPT I.10: Vẽ sequence diagram chuẩn bị cho JavaFX Controller gọi Service ở Part B.

8.3. NHÓM II – 10 prompt kỹ thuật nâng cao

PROMPT II.1: So sánh dirty checking, merge và bulk HQL update cho UPDATE; chỉ rõ lifecycle/caching trade-off.

PROMPT II.2: Phân tích race condition giữa pre-check unique email và DB UNIQUE constraint; đề xuất robust error handling.

PROMPT II.3: Phân tích transaction isolation ảnh hưởng hai concurrent UPDATE cùng Student như thế nào.

PROMPT II.4: Thiết kế optimistic locking `@Version` cho Student và test lost-update.

PROMPT II.5: Phân tích batch INSERT 10k Students với IDENTITY vs sequence-like strategies và flush/clear cadence.

PROMPT II.6: Review exception handling DAO và tìm cases rollback/close không an toàn.

PROMPT II.7: Phân tích detached entity overposting: form gửi stale/full entity rồi merge có thể overwrite field nào.

PROMPT II.8: Thiết kế query/search/pagination API tránh load toàn bộ Student list.

PROMPT II.9: Phân tích first-level cache stale data khi bulk mutation chạy trong cùng persistence context.

PROMPT II.10: Review module-info/JPA reflection requirements và giải thích `opens` packages khi JPMS được dùng.

8.4. NHÓM III – 10 prompt production engineering

PROMPT III.1: Refactor DAO-per-method transaction sang Service transaction boundary cho multi-step use case.

PROMPT III.2: Thiết kế configuration strategy dev/test/prod với environment variables/secrets manager.

PROMPT III.3: Chuyển `hbm2ddl=update` sang Flyway/Liquibase + Hibernate validate; lập deployment workflow.

PROMPT III.4: Thiết kế integration-test harness cho CRUD bằng SQL Server container/test instance và deterministic cleanup.

PROMPT III.5: Thiết kế observability CRUD: slow queries, transaction latency, connection pool, error categories.

PROMPT III.6: Thiết kế repository API cho 1 triệu Students: pagination, filtering, projections, indexes.

PROMPT III.7: Thiết kế concurrency policy cho update marks/profile, optimistic locking và retry UX.

PROMPT III.8: Thiết kế audit trail cho create/update/delete mà không nhét audit logic vào DAO.

PROMPT III.9: Threat-model persistence layer: SQL/HQL injection, credentials, sensitive fields, mass assignment, logs.

PROMPT III.10: Review composition root/DI để JavaFX Controller không new Repository/DAO trực tiếp.

8.5. NHÓM IV – 10 prompt current ecosystem / trends

PROMPT IV.1: Kiểm tra latest stable Hibernate ORM, release date và compatibility Java/Jakarta Persistence.

PROMPT IV.2: Đọc official Hibernate 7.4 User Guide phần Session/transactions và so sánh course DAO flow.

PROMPT IV.3: Khảo sát Hibernate Data Repositories/Jakarta Data 1.0 và so sánh với manual Repository.

PROMPT IV.4: Khảo sát Hibernate Processor/type-safe query tooling cho repository/query refactoring.

PROMPT IV.5: Kiểm tra Hibernate ORM 8.0 development status và Jakarta Persistence 4.0; phân loại WATCH LATER.

PROMPT IV.6: Khảo sát schema tooling recommendation hiện tại: Hibernate schema generation vs migration tools.

PROMPT IV.7: Khảo sát batching/current performance features trong Hibernate 7.4 cho high-volume CRUD.

PROMPT IV.8: Khảo sát modern HQL mutation/query API thay đổi so với course legacy examples.

PROMPT IV.9: Khảo sát observability integrations hiện tại của Hibernate/JFR/statistics/Micrometer ecosystem.

PROMPT IV.10: Tạo '2026 Hibernate CRUD Production Radar': ORM 7.4, Jakarta Persistence 3.2, repositories, migrations, locking, batching, observability.

8.6. AI Verification Worksheet

Claim	Nguồn	Experiment	Kết luận
Dirty checking	Hibernate docs + SQL log	managed change	
Merge semantics	docs + debugger	detached update	
Rollback atomicity	DB transaction	intentional fail	
Latest Hibernate	hibernate.org official	dependency check	
Layering	source imports/call graph	architecture trace	

IX. SINH VIÊN CẦN TỰ KIỂM TRA GÌ SAU BUỔI HỌC

□ 1. Tôi vẽ được CRUD application layers.
□ 2. Tôi giải thích Hibernate config/bootstrap flow.
□ 3. Tôi biết SessionFactory khác Session.
□ 4. Tôi viết CREATE có transaction/rollback.
□ 5. Tôi viết READ by id/list/search bằng typed query.
□ 6. Tôi phân biệt managed dirty checking và detached merge.
□ 7. Tôi viết DELETE và kiểm FK/association.
□ 8. Tôi dùng Optional/not-found contract rõ.
□ 9. Tôi đặt validation ở Service và constraint ở DB.
□ 10. Tôi không cho UI/Main dùng Hibernate Session trực tiếp.
□ 11. Tôi hiểu Repository che DAO/Hibernate details.
□ 12. Tôi kiểm DB/SQL sau mỗi CRUD.
□ 13. Tôi chứng minh rollback không partial commit.
□ 14. Tôi biết vì sao hbm2ddl update chỉ phù hợp lab/dev.
□ 15. Tôi sẵn sàng nối JavaFX Controller vào Service ở Part B.

IX.1. Exit ticket

Câu hỏi	Trả lời
Khi entity đang managed, UPDATE có cần gọi `update()` không?	
`merge()` trả về object gì?	
Vì sao pre-check duplicate không thay thế UNIQUE constraint?	
Transaction nên nằm đâu với multi-step use case?	
JavaFX Controller ở Part B nên gọi layer nào?	

TÀI LIỆU THAM CHIẾU & SOURCE TRACEABILITY

- Course source: Chapter 18 – Hibernate Application Development & CRUD Operations, pages 1–20 for Part A: project structure, Student/Book, StudentDAO CRUD, Repository, Service, module-info, hibernate.cfg.xml.
- Course source: Chapter 17/Lab 07 baseline used only to preserve continuity of SessionFactory/DAO/Repository/Service architecture.
- Official Hibernate ORM releases (accessed 2026-10-12): Hibernate ORM 7.4 latest stable; 7.4.13.Final released 2026-10-11; 8.0 development.
- Official Hibernate ORM 7.4 compatibility: Java 17/21/25/26, Jakarta Persistence 3.2.

VERSION: Current ecosystem is explicitly separated from course baseline. Do not upgrade the teaching project dependency simply because a newer Hibernate release exists.